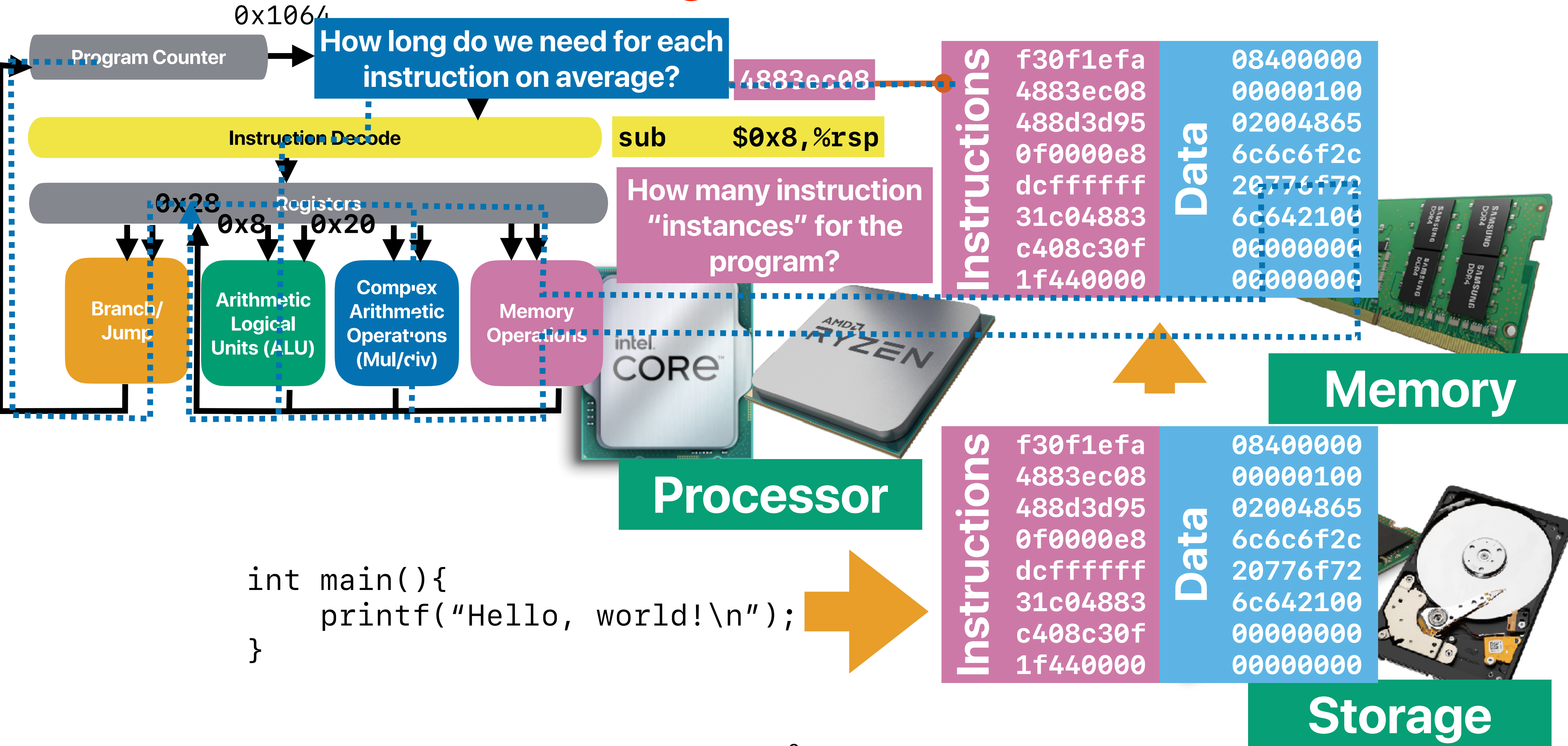


Performance (2): Do the right thing

Hung-Wei Tseng

Execution time of a program in the von Neumann model



Classic CPU Performance Equation (ET of a program)

How many instruction
"instances" for the
program?

×

How long do we need for each
instruction on average?

$$\text{Execution Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

C Code	x86 instructions
<pre>int init_data(int64_t *data, int data_size) { register unsigned int i = 0; for(i = 0; i < data_size; i++) { s+=data[i]; } return s; } int main(int argc, char **argv) { int *data = malloc(8000000000); init_data(data, 1000000000) return 0; }</pre>	<pre>init_data: .LFB16: endbr64 testl %esi, %esi jle .L2 leal -1(%rsi), %ecx xorq %rax, %rax .L3: movslq (%rdi), %rdx addq \$4, %rdi addq %rdx, %rax cmpq %rcx, %rdi jne .L3 .L2: xorlq %rax, %rax ret</pre>

If data memory access instructions takes 5 cycles,
branch 2 cycles, others take only 1 cycle, CPU freq. = 4 GHz

$$CPI_{average} = 20\% \times 5 + 20\% \times 2 + 60\% \times 1 = 2$$

memory inst.

$$ET = (5 \times 10^9) \times 2 \times \frac{1}{4 \times 10^9} \text{ sec} = 2.5 \text{ sec}$$

←branch inst.

Is TFLOPS (Tera Floating-point Operations Per Second) a good metric?

$$\begin{aligned} TFLOPS &= \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Execution Time}} \\ &= \frac{IC \times \% \text{ of floating point instructions} \times 10^{-12}}{IC \times CPI \times CT} \\ &= \frac{\% \text{ of floating point instructions} \times 10^{-12}}{CPI \times CT} \end{aligned} \quad \text{IC is gone!}$$

- If we have more iterations? Larger datasets? — potentially changes the IC
- What if the hardware trade (cheat) performance with accuracy?
- Cannot compare different ISA/compiler
 - What if the compiler can generate code with fewer instructions?
 - What if new architecture has more IC but also lower CPI?
- If floating point operations are not critical in the target application?

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

$O(n^2)$

Complexity

$O(n^2)$

Same

Instruction Count?

Same

Same

Clock Rate

Same

Better

CPI

Worse

How compilers affect performance

- If we turn on "-O3" in gcc when compiling both code snippets **A** and **B**, how many of the following can we expect?

①  Compiler optimizations can reduce IC for both
Compiler can apply loop unrolling, constant propagation naively to reduce IC

② Compiler optimizations can make the CPI lower for both
Reduced IC does not necessarily mean lower CPI — compiler may pick one longer instruction to replace a few shorter ones

③ Compiler optimizations can make the ET lower for both
Compiler cannot guarantee the combined effects lead to better performance!

④ Compiler optimizations can transform code B into code A
Compiler will not significantly change programmer's code since compiler cannot guarantee if doing that would affect the correctness

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Programmers can control all three factors in the classic performance equation when composing the program
- Compiler optimization does not always help
 - Compiler optimizes code based on some assumptions that may not be true on all computers
 - Programmers' can write code in a way facilitating optimizations!

What matters to the classical CPU performance equation?

$$Performance = \frac{1}{Execution\ Time}$$

$$Execution\ Time = \frac{Instructions}{Program} \times \frac{Cycles}{Instruction} \times \frac{Seconds}{Cycle}$$

$$ET = IC \times CPI \times CT$$

- IC (Instruction Count)
 - ISA, Compiler, algorithm, programming language, **programmer**
- CPI (Cycles Per Instruction)
 - Machine Implementation, microarchitecture, compiler, application, algorithm, programming language, **programmer**
- Cycle Time (Seconds Per Cycle)
 - Process Technology, microarchitecture, **programmer**

3:00

**If there was one right thing that
you've done, what & why is that?**

Outline

- What does better mean?
- Amdahl's Law and its implications

Quantitative Analysis of “Better”



Speedup of Y over X

- Consider the same program on the following two machines, X and Y. By how much Y is faster than X?

	Clock Rate	Dynamic Instruction Count	Percentage of Type-A	CPI of Type-A	Percentage of Type-B	CPI of Type-B	Percentage of Type-C	CPI of Type-C
Machine X	4 GHz	5000000000	20%	5	20%	2	60%	1
Machine Y	6 GHz	5000000000	20%	7	20%	2	60%	1

- A. 0.2
- B. 0.25
- C. 0.8
- D. 1.25
- E. No changes



Speedup

- The relative performance between two machines, X and Y. Y is n times faster than X

$$n = \frac{\textit{Execution Time}_X}{\textit{Execution Time}_Y}$$

- The speedup of Y over X

$$\textit{Speedup} = \frac{\textit{Execution Time}_X}{\textit{Execution Time}_Y}$$

Speedup of Y over X

- Consider the same program on the following two machines, X and Y. By how much Y is faster than X?

	Clock Rate	Dynamic Instruction	Percentage of Type-A	CPI of Type-A	Percentage of Type-B	CPI of Type-B	Percentage of Type-C	CPI of Type-C
Machine X	4 GHz	5000000000	20%	5	20%	2	60%	1
Machine Y	6 GHz	5000000000	20%	7	20%	2	60%	1

A. 0.2

B. 0.25

C. 0.8

D. 1.25

E. No changes

$$ET_X = (5 \times 10^9) \times (20\% \times 5 + 20\% \times 2 + 60\% \times 1) \times \frac{1}{4 \times 10^9} \text{ sec} = 2.5 \text{ sec}$$

$$ET_Y = (5 \times 10^9) \times (20\% \times 7 + 20\% \times 2 + 60\% \times 1) \times \frac{1}{6 \times 10^9} \text{ secs} = 2 \text{ secs}$$

$$\text{Speedup} = \frac{\text{Execution Time}_X}{\text{Execution Time}_Y} = \frac{2.5}{2} = 1.25$$

Takeaways: What is and what affects “performance”?

- The only definition of Y is Speedup times faster than X —

$$Speedup = \frac{Execution\ Time_X}{Execution\ Time_Y}$$

COVER FEATURE

Amdahl's Law in the

Amdahl's Law — and It's

Implication in the Multicore Era

Mark D. Hill, University of Wisconsin-Madison
Michael R. Marty, Google

Augmenting Amdahl's law with a corollary for multicore hardware makes it relevant to future generations of chips with multiple processor cores. Obtaining optimal multicore performance will require further research in both extracting more parallelism and making sequential cores faster.

Mark D. Hill, University of Wisconsin-Madison

Michael R. Marty, Google

In IEEE Computer, vol. 41, no. 7

Amdahl's Law

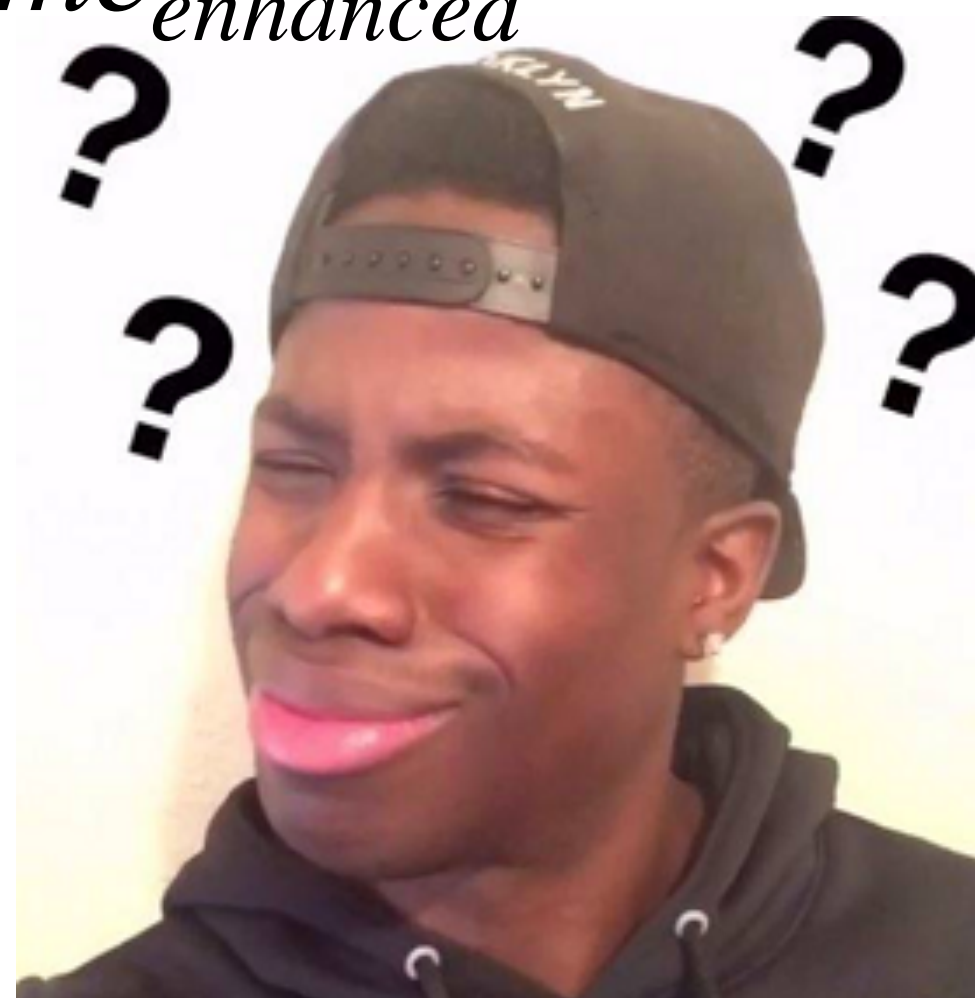


$$Speedup_{enhanced}(f, s) = \frac{1}{(1 - f) + \frac{f}{s}}$$

f — The fraction of time in the original program

s — The speedup we can achieve on f

$$Speedup_{enhanced} = \frac{Execution\ Time_{baseline}}{Execution\ Time_{enhanced}}$$



Amdahl's Law

$$Speedup_{enhanced}(f, s) = \frac{1}{(1 - f) + \frac{f}{s}}$$

Execution Time_{baseline} = 1

baseline



enhanced

Execution Time_{enhanced} = $(1 - f) + \frac{f}{s}$



$$Speedup_{enhanced} = \frac{Execution\ Time_{baseline}}{Execution\ Time_{enhanced}} = \frac{1}{(1 - f) + \frac{f}{s}}$$

FINAL FANTASY XV

POCKET EDITION **HD**

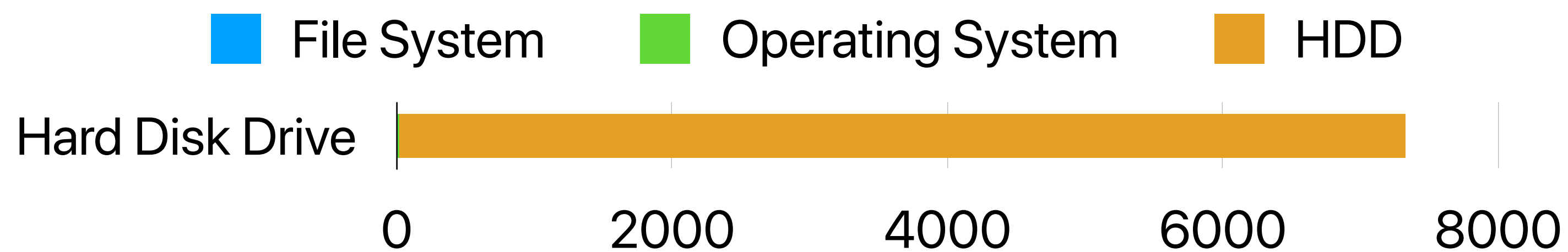




Practicing Amdahl's Law

- Final Fantasy XV spends lots of time loading a map — within which period that 95% of the time on the accessing the H.D.D., the rest in the operating system, file system and the I/O protocol. If we replace the H.D.D. with a flash drive, which provides 100x faster access time. By how much can we speed up the map loading process?

- A. $\sim 7x$
- B. $\sim 10x$
- C. $\sim 17x$
- D. $\sim 29x$
- E. $\sim 100x$



Latency (us)

A

B

C

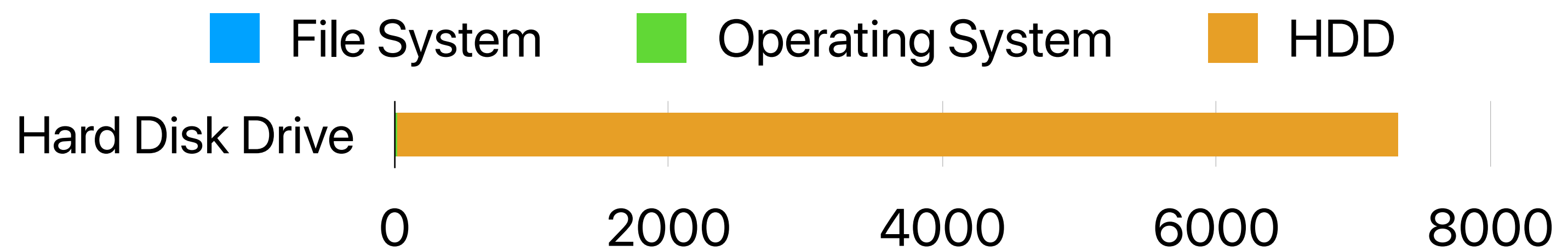
D

E

Practicing Amdahl's Law

- Final Fantasy XV spends lots of time loading a map — within which period that 95% of the time on the accessing the H.D.D., the rest in the operating system, file system and the I/O protocol. If we replace the H.D.D. with a flash drive, which provides 100x faster access time. By how much can we speed up the map loading process?

- A. ~7x
- B. ~10x
- C. ~17x**
- D. ~29x
- E. ~100x



$$Speedup_{enhanced}(95\%, 100) = \frac{1}{(1 - 95\%) + \frac{95\%}{100}} = 16.81 \times$$

Amdahl's Law on Multiple Optimizations

- We can apply Amdahl's law for multiple optimizations
- These optimizations must be dis-joint!
 - If optimization #1 and optimization #2 are dis-joint:



$$Speedup_{enhanced}(f_{Opt1}, f_{Opt2}, s_{Opt1}, s_{Opt2}) = \frac{1}{(1 - f_{Opt1} - f_{Opt2}) + \frac{f_{Opt1}}{s_{Opt1}} + \frac{f_{Opt2}}{s_{Opt2}}}$$

- If optimization #1 and optimization #2 are not dis-joint:



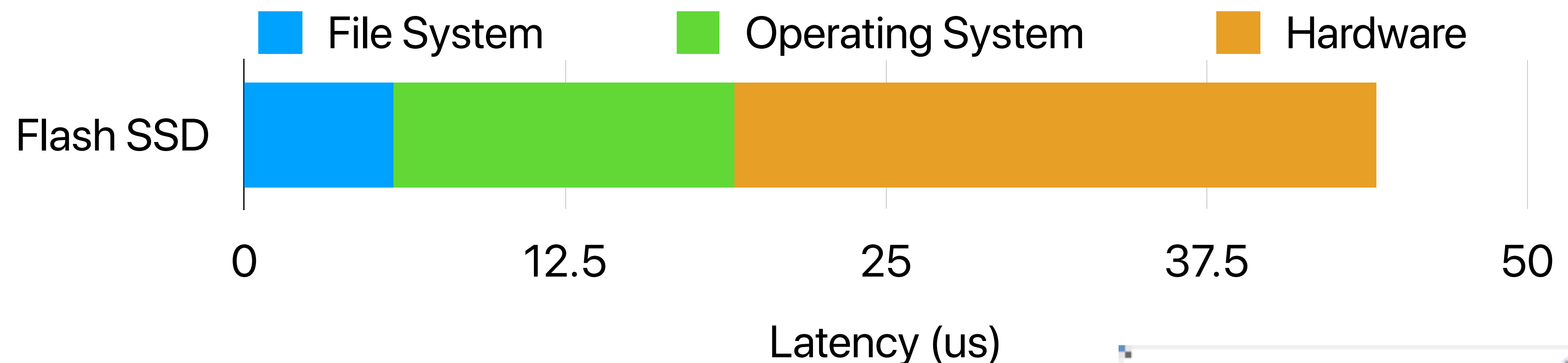
$$Speedup_{enhanced}(f_{OnlyOpt1}, f_{OnlyOpt2}, f_{BothOpt1Opt2}, s_{OnlyOpt1}, s_{OnlyOpt2}, s_{BothOpt1Opt2}) = \frac{1}{(1 - f_{OnlyOpt1} - f_{OnlyOpt2} - f_{BothOpt1Opt2}) + \frac{f_{BothOpt1Opt2}}{s_{BothOpt1Opt2}} + \frac{f_{OnlyOpt1}}{s_{OnlyOpt1}} + \frac{f_{OnlyOpt2}}{s_{OnlyOpt2}}}$$



Speedup further!

- With the latest flash memory technologies, the system spends 16% of time on accessing the flash, and the software overhead is now 84%. If your company ask you and your team to invent a new memory technology that replaces flash to achieve 2x speedup on loading maps, how much faster the new technology needs to be?

- A. ~5x
- B. ~10x
- C. ~20x
- D. ~100x
- E. None of the above

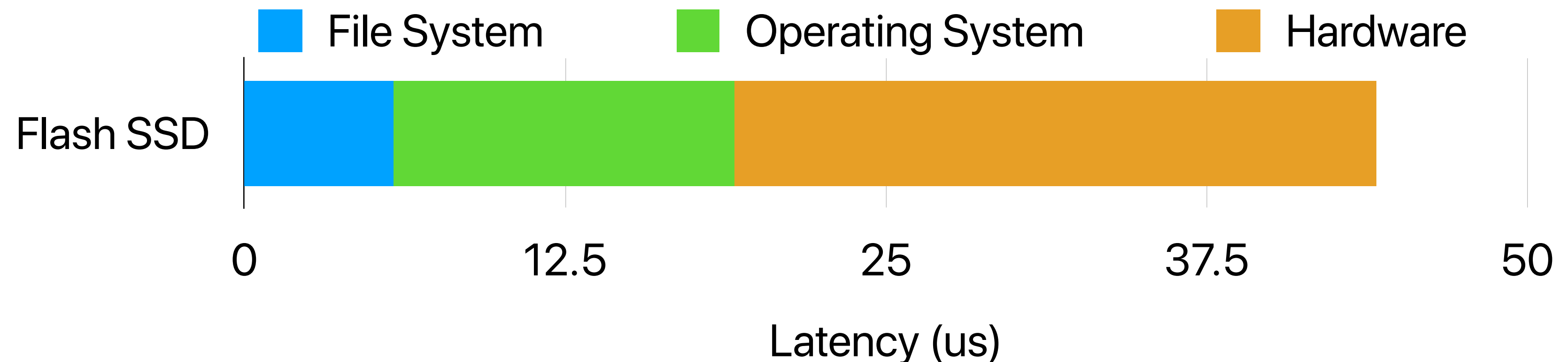


Speedup further!

- With the latest flash memory technologies, the system spends 16% of time on accessing the flash, and the software overhead is now 84%. If your company ask you and your team to invent a new memory technology that replaces flash to achieve 2x speedup on loading maps, how much faster the new technology needs to be?

- A. ~5x
- B. ~10x
- C. ~20x
- D. ~100x

E. None of the above



$$Speedup_{enhanced}(16\%, x) = \frac{1}{(1 - 16\%) + \frac{16\%}{x}} = 2$$

Does this make sense?

$$x = 0.47$$

Amdahl's Law Corollary #1

- The maximum speedup is bounded by

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f) + \frac{f}{\infty}}$$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

Speedup further!

- With the latest flash memory technologies, the system spends 16% of time on accessing the flash, and the software overhead is now 84%. If your company ask you and your team to invent a new memory technology that replaces flash to achieve 2x speedup on loading maps, how much faster the new technology needs to be?

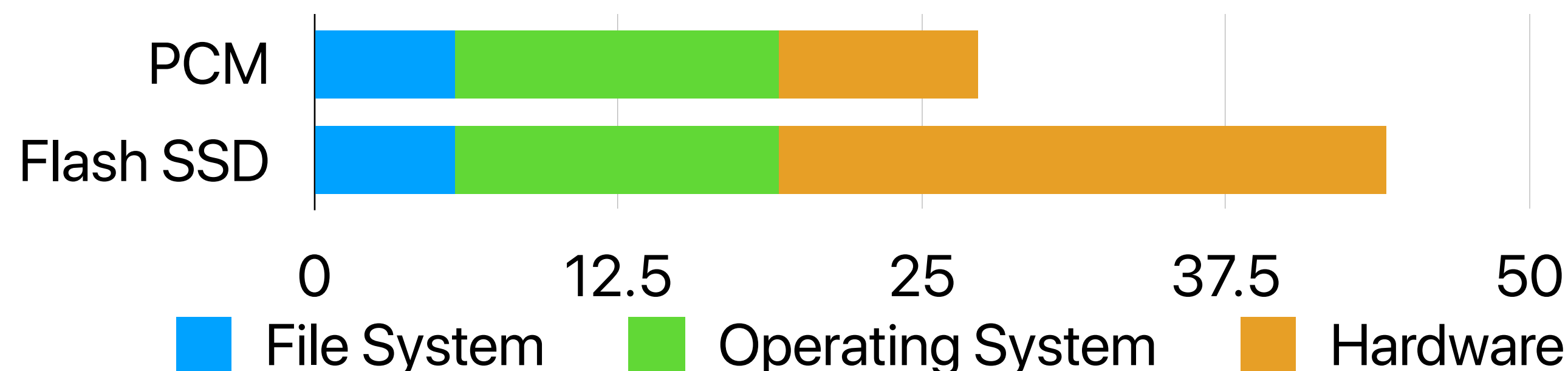
A. ~5x

B. ~10x

C. ~20x

D. ~100x

E. None of the above



$$Speedup_{max}(16\%, \infty) = \frac{1}{(1 - 16\%)} = 1.19$$

2x is not possible

NEWS

Intel kills the remnants of Optane memory

The speed-boosting storage tech was already on the ropes.



By [Michael Crider](#)

Staff Writer, PCWorld | JUL 29, 2022 6:59 AM PDT

You spent a lot of money but can only get 20% performance gain!

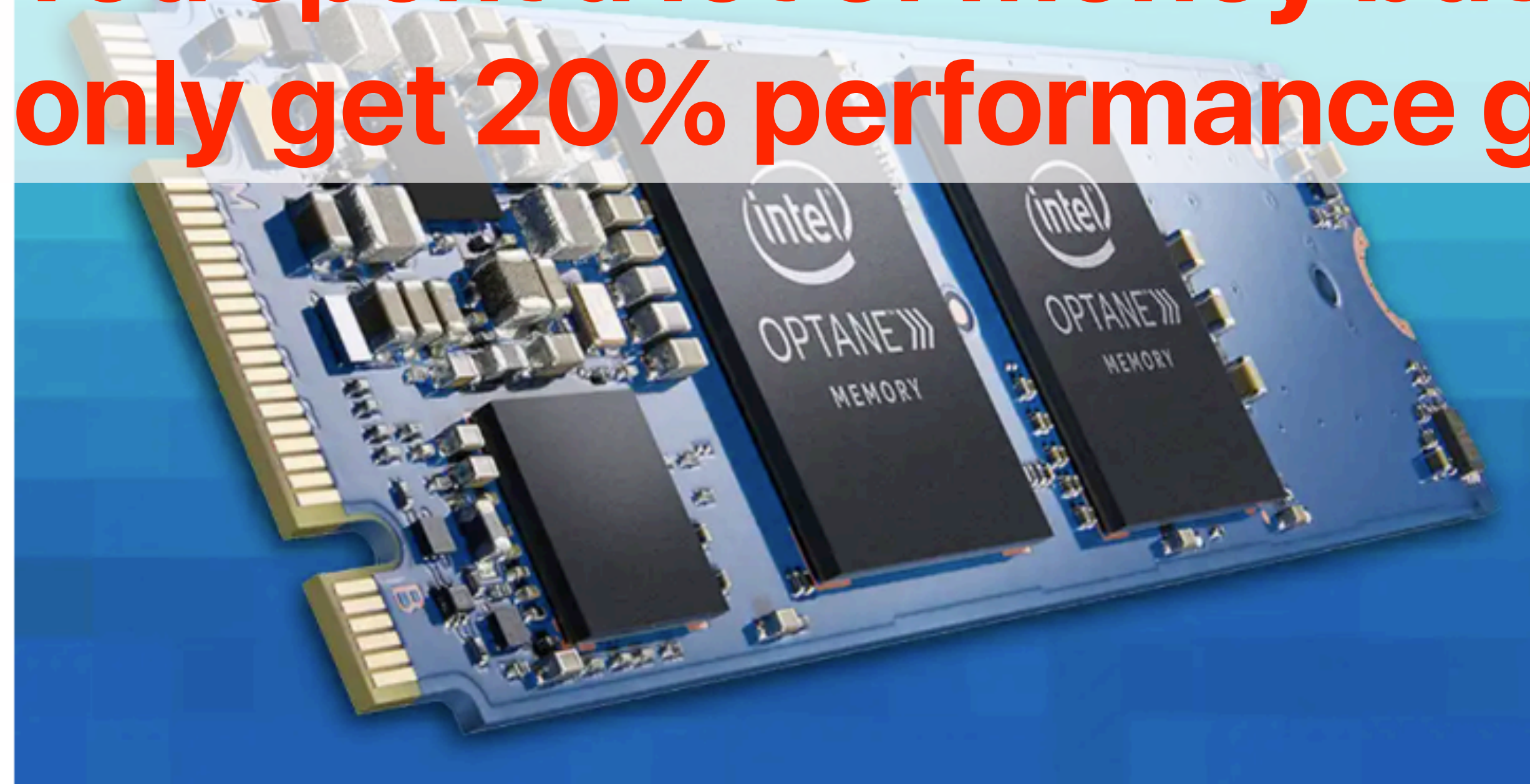


Image: Intel

MILLIPOR
SIGMA



MISSION® es

targeting mou
2010204k13r



Practicing Amdahl's Law (2)

- After applying an SSD, Final Fantasy XV now spends 16% in accessing SSD, the rest in the operating system, file system and the I/O protocol. Which of the following proposals would give as the largest performance gain?
 - A. Replacing the CPU to speed up the rest by 2x
 - B. Replacing the CPU to speed up the rest by 1.2x and replacing the SSD to speed up the SSD part by 100x
 - C. Replacing the CPU to speed up the rest by 1.5x and replacing the SSD to speed up the SSD part by 20x
 - D. Replacing the SSD to speed up the SSD part by 200x
 - E. They are about the same

A screenshot of a Pollev poll interface. It shows five horizontal input fields labeled A, B, C, D, and E. The fields are currently empty, suggesting a multiple-choice or open-ended poll. The interface is clean and modern, with a light blue header and footer.

Practicing Amdahl's Law (2)

- After applying an SSD, Final Fantasy XV now spends 16% in accessing SSD, the rest in the operating system, file system and the I/O protocol. Which of the following proposals would give as the largest performance gain?

$$Speedup_{enhanced}(84\%, 16\%, 2, 1) = \frac{1}{(1 - 16\%) + \frac{84\%}{2}} = 1.72 \times$$

A. Replacing the CPU to speed up the rest by 2x

B. Replacing the CPU to speed up the rest by 1.2x and replacing the SSD to speed up the SSD part by 100x

$$Speedup_{enhanced}(84\%, 16\%, 1.2, 100) = \frac{1}{(1 - 84\% - 16\%) + \frac{84\%}{1.2} + \frac{16\%}{100}} = 1.43 \times$$

C. Replacing the CPU to speed up the rest by 1.5x and replacing the SSD to speed up the SSD part by 20x

$$Speedup_{enhanced}(84\%, 16\%, 1.5, 20) = \frac{1}{(1 - 84\% - 16\%) + \frac{84\%}{1.5} + \frac{16\%}{20}} = 1.76 \times$$

D. Replacing the SSD to speed up the SSD part by 200x

E. They are about the same

$$Speedup_{enhanced}(84\%, 16\%, 1, 200) = \frac{1}{(1 - 16\%) + \frac{16\%}{200}} = 1.19 \times$$

If we don't touch the 84% part, it's hard to speedup!

Corollary #1 on Multiple Optimizations

- If we can pick just one thing to work on/optimize



$$Speedup_{max}(f_1, \infty) = \frac{1}{(1 - f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1 - f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1 - f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1 - f_4)}$$

The biggest f_x would lead to the largest *Speedup*_{max}!

Corollary #2 — make the common case fast!

- When f is small, optimizations will have little effect.
- Common == **most time consuming** not necessarily the most frequent
- The uncommon case doesn't make much difference
- The common case can change based on inputs, compiler options, optimizations you've applied, etc.

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

- Corollary 1 — each optimization has an upper bound
- Corollary 2 — make the common case (the most time consuming case) fast!

$$\begin{aligned} Speedup_{max}(f_1, \infty) &= \frac{1}{(1-f_1)} \\ Speedup_{max}(f_2, \infty) &= \frac{1}{(1-f_2)} \\ Speedup_{max}(f_3, \infty) &= \frac{1}{(1-f_3)} \\ Speedup_{max}(f_4, \infty) &= \frac{1}{(1-f_4)} \end{aligned}$$

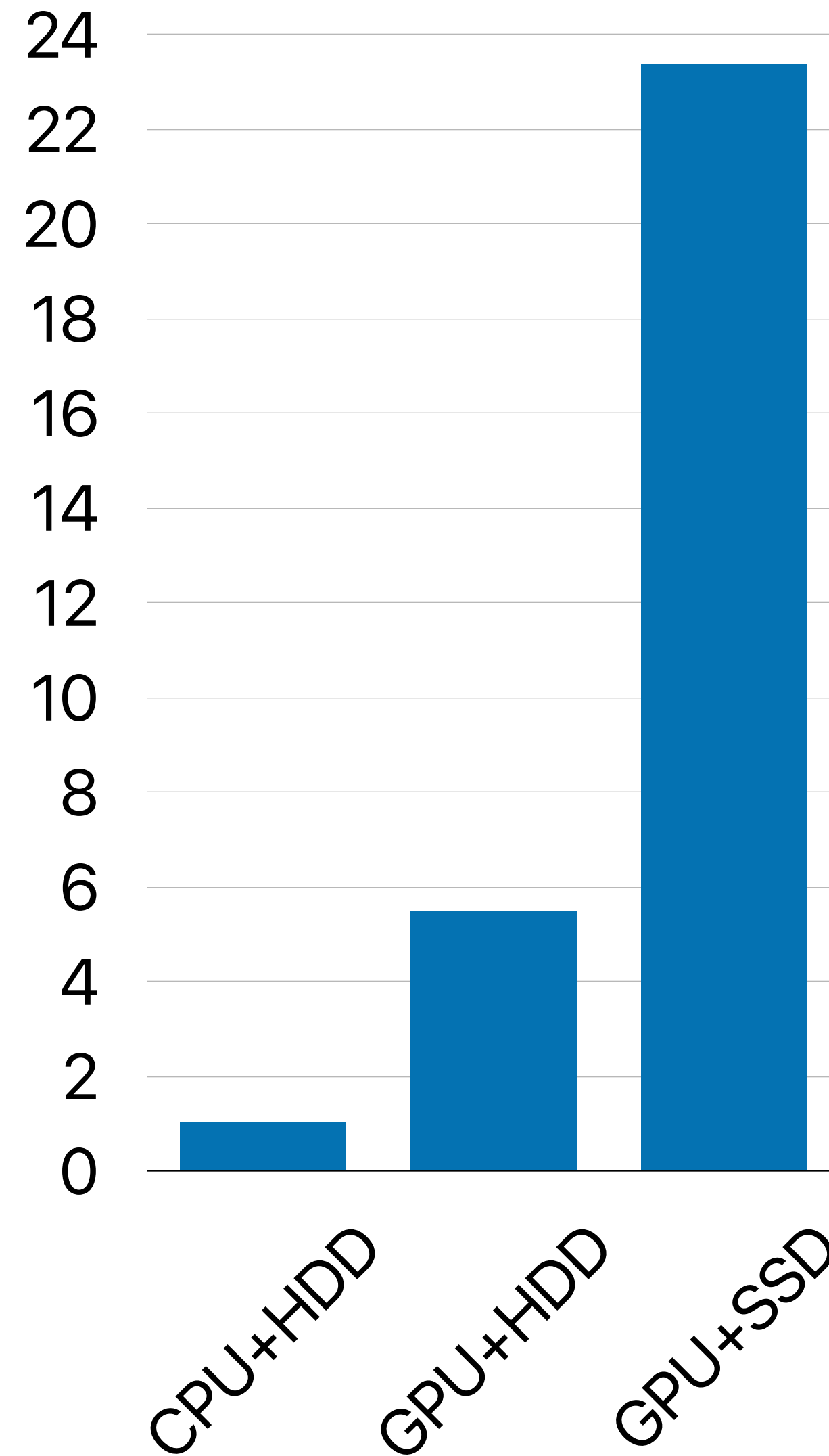
Identify the most time consuming part

- Use perf
 - perf record command
 - perf report —stdio
- Use prof
 - Compile your program with -pg flag
 - Run the program
 - It will generate a gmon.out
 - gprof your_program gmon.out > your_program.prof
 - It will give you the profiled result in your_program.prof
- Or insert timestamps

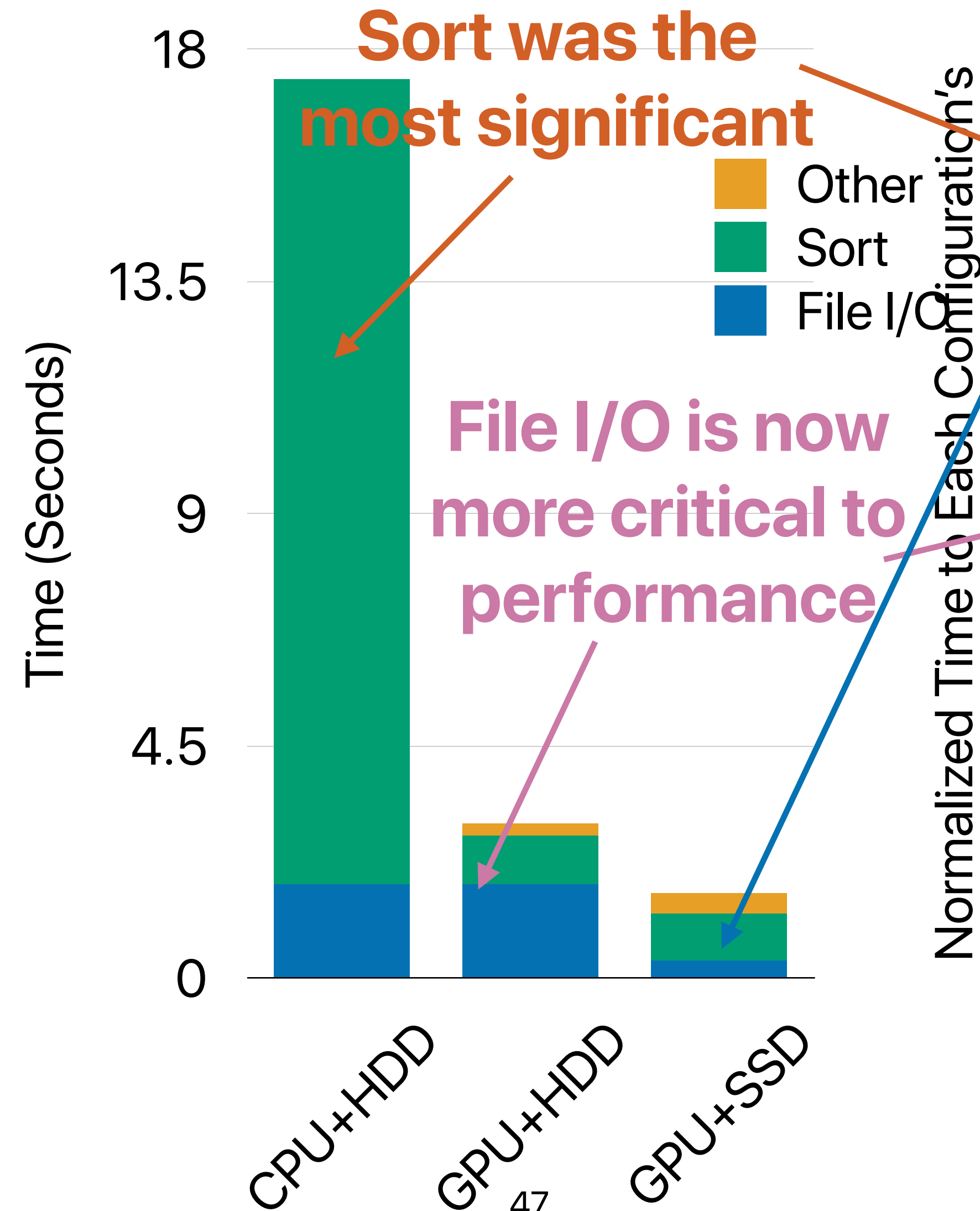
Demo — sort

Something else (e.g., data movement) matters more

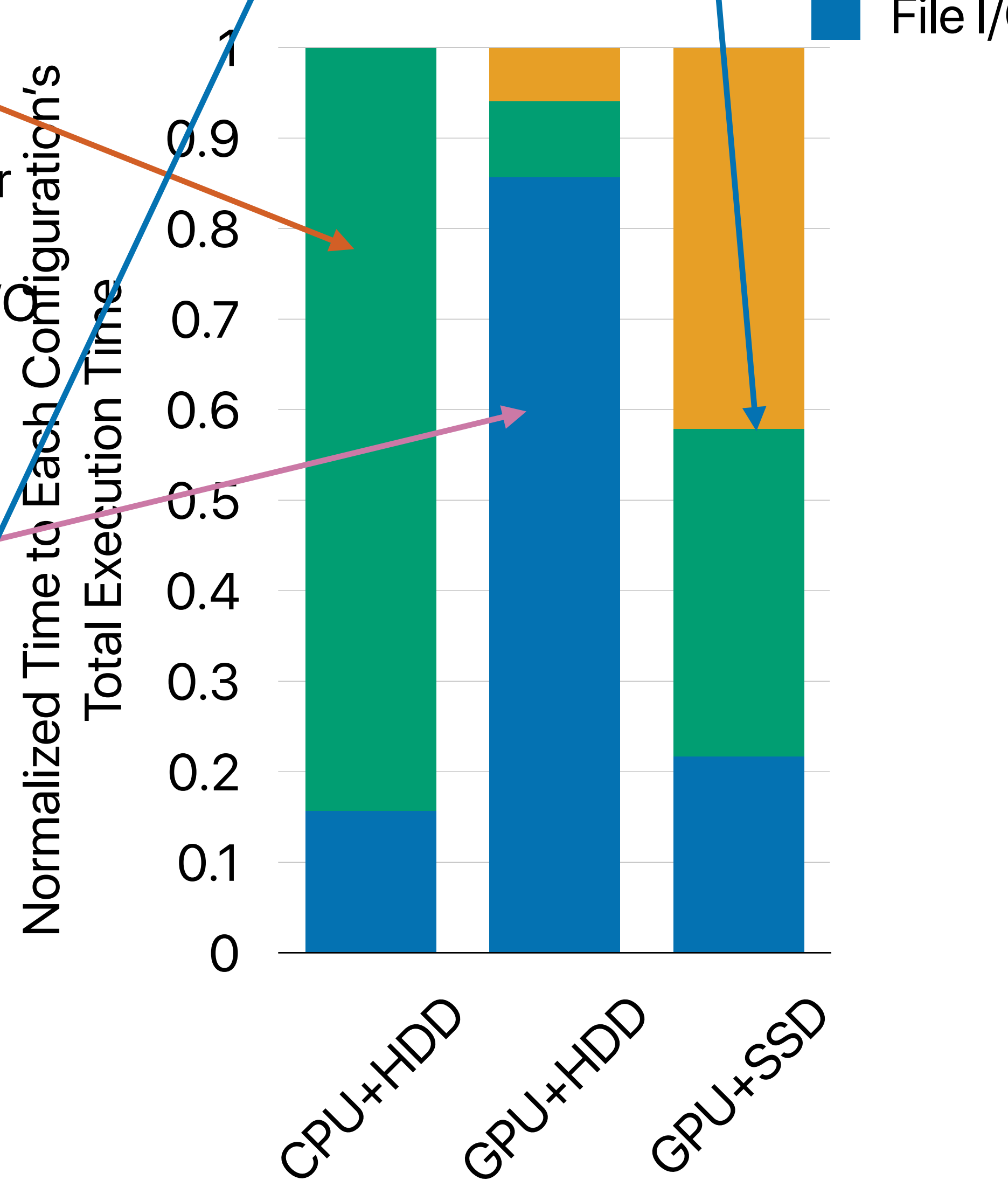
Speedup



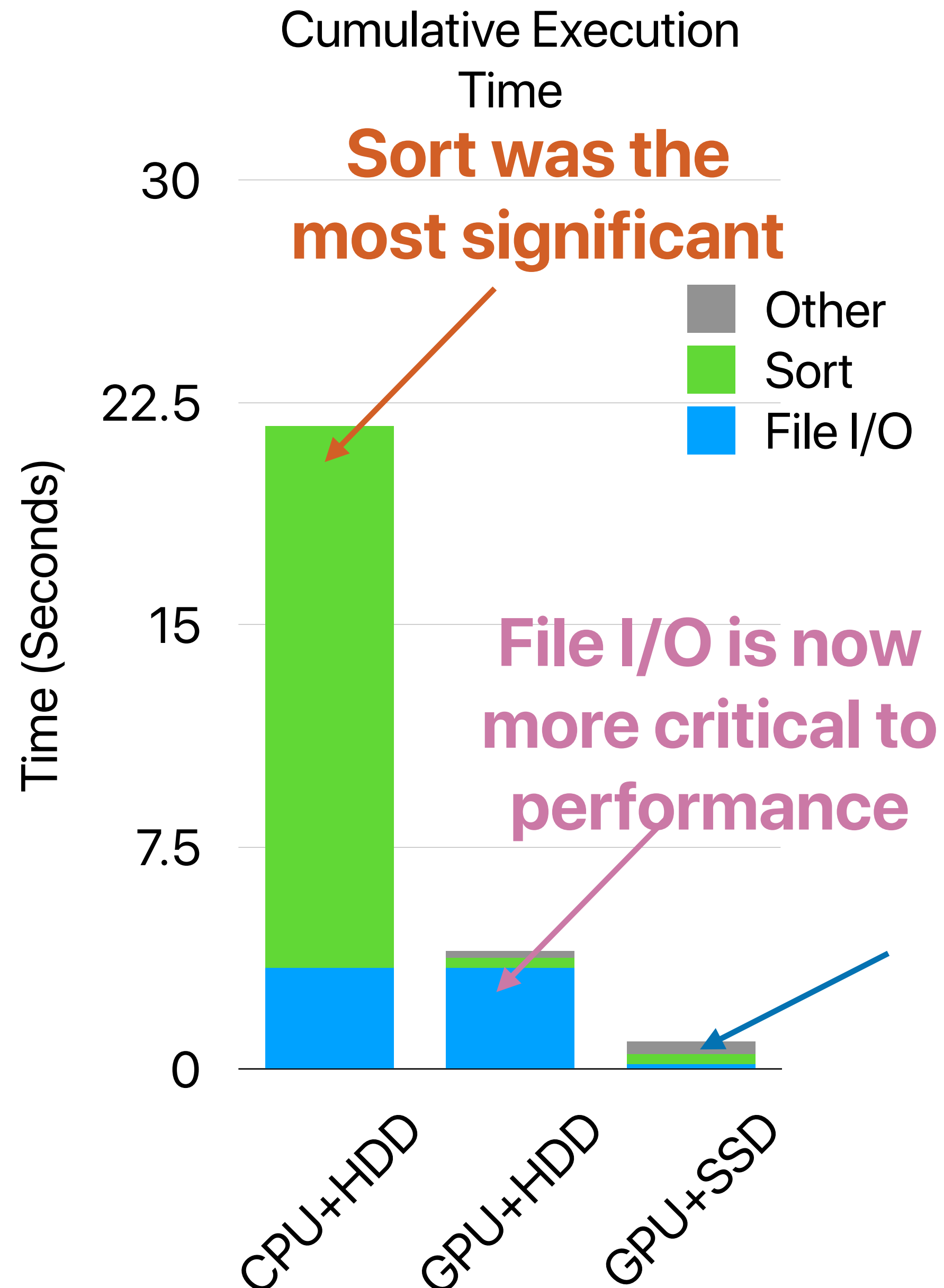
Cumulative Execution Time



Execution Time Breakdown



Corollary #3: optimization has a moving target



- With optimization, the common becomes uncommon.
- An uncommon case will (hopefully) become the new common case.
- Now you have a new target for optimization — You have to revisit "Amdahl's Law" every time you applied some optimization

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

- Corollary 1 — each optimization has an upper bound
- Corollary 2 — make the common case (the most time consuming case) fast!
- Corollary 3 — Optimization has a moving target

$$\begin{aligned} Speedup_{max}(f_1, \infty) &= \frac{1}{(1-f_1)} \\ Speedup_{max}(f_2, \infty) &= \frac{1}{(1-f_2)} \\ Speedup_{max}(f_3, \infty) &= \frac{1}{(1-f_3)} \\ Speedup_{max}(f_4, \infty) &= \frac{1}{(1-f_4)} \end{aligned}$$

Amdahl's Law on Multicore Architectures

- Symmetric multicore processor with n cores (if we assume the processor performance scales perfectly)

$$Speedup_{parallel}(f_{parallelizable}, n) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{n}}$$



Amdahl's Law on Multicore Architectures

- Regarding Amdahl's Law on multicore architectures, how many of the following statements is/are correct?
 - ① If we have unlimited parallelism, the performance of executing each parallel partition does not matter as long as the performance slowdown in each piece is bounded
 - ② With unlimited amount of parallel hardware units, single-core performance does not matter anymore
 - ③ With unlimited amount of parallel hardware units, the maximum speedup will be bounded by the fraction of parallel parts
 - ④ With unlimited amount of parallel hardware units, the effect of scheduling and data exchange overhead is minor

A. 0
B. 1
C. 2
D. 3
E. 4

A screenshot of a Pollev poll interface. It shows a list of five answer options, labeled A through E, each in a separate input field. The options are: A, B, C, D, and E. The interface is clean and modern, with a light blue header and a white background for the options.

Amdahl's Law on Multicore Architectures

- Regarding Amdahl's Law on multicore architectures, how many of the following statements is/are correct?

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable} \times Speedup(< 1)}{\infty}}$$

① ✓ If we have unlimited parallelism, the performance of executing each parallel partition does not matter as long as the performance slowdown in each piece is bounded

② With unlimited amount of parallel hardware units, single-core performance does not matter anymore

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable})} \quad \text{speedup is determined by } 1-f$$

③ ✓ With unlimited amount of parallel hardware units, the maximum speedup will be bounded by the fraction of parallel parts

④ With unlimited amount of parallel hardware units, the effect of scheduling and data exchange overhead is minor

A. 0

B. 1

C. 2

D. 3

E. 4

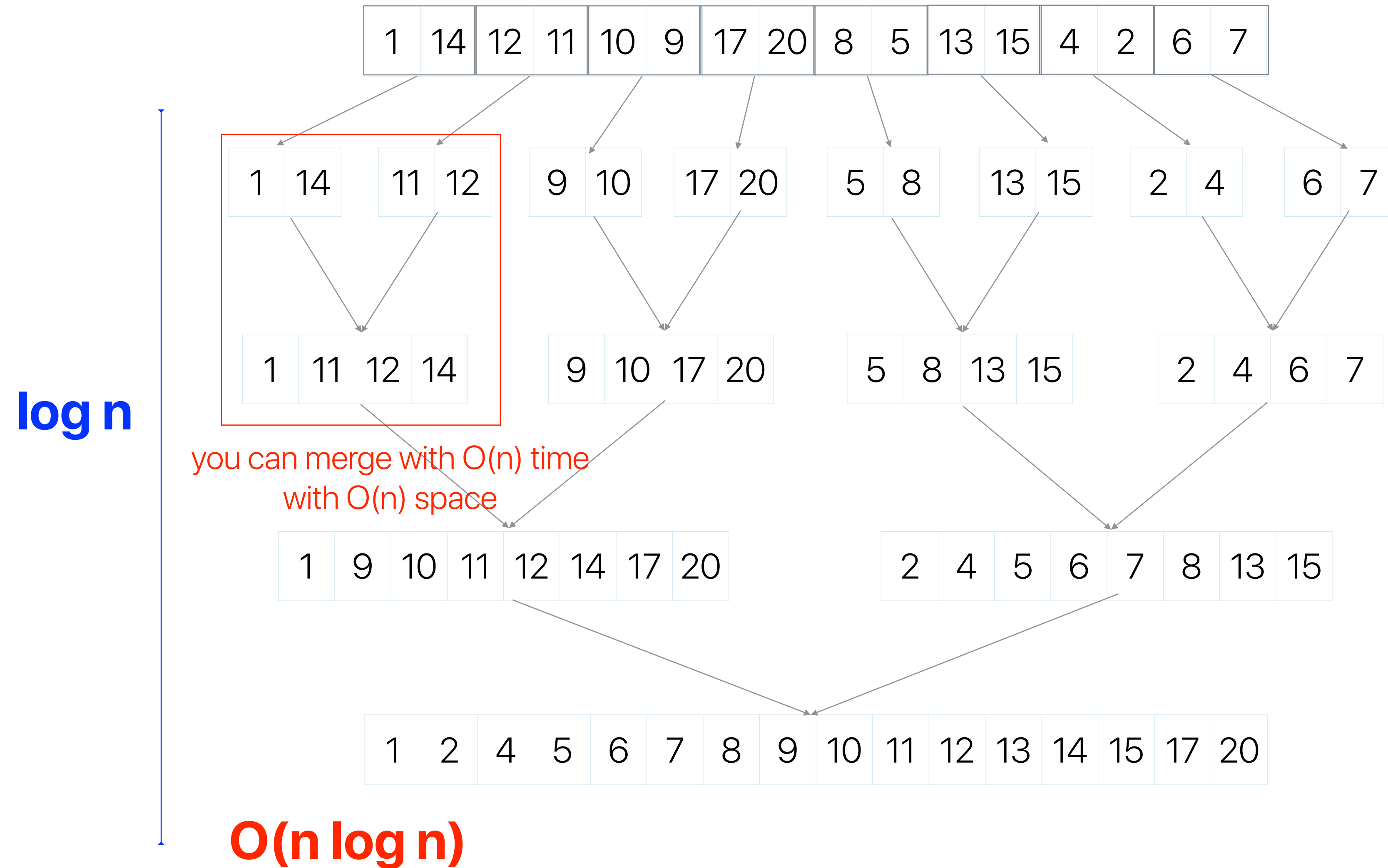
Demo — merge sort v.s. bitonic sort on GPUs

Merge Sort
 $O(n \log_2 n)$

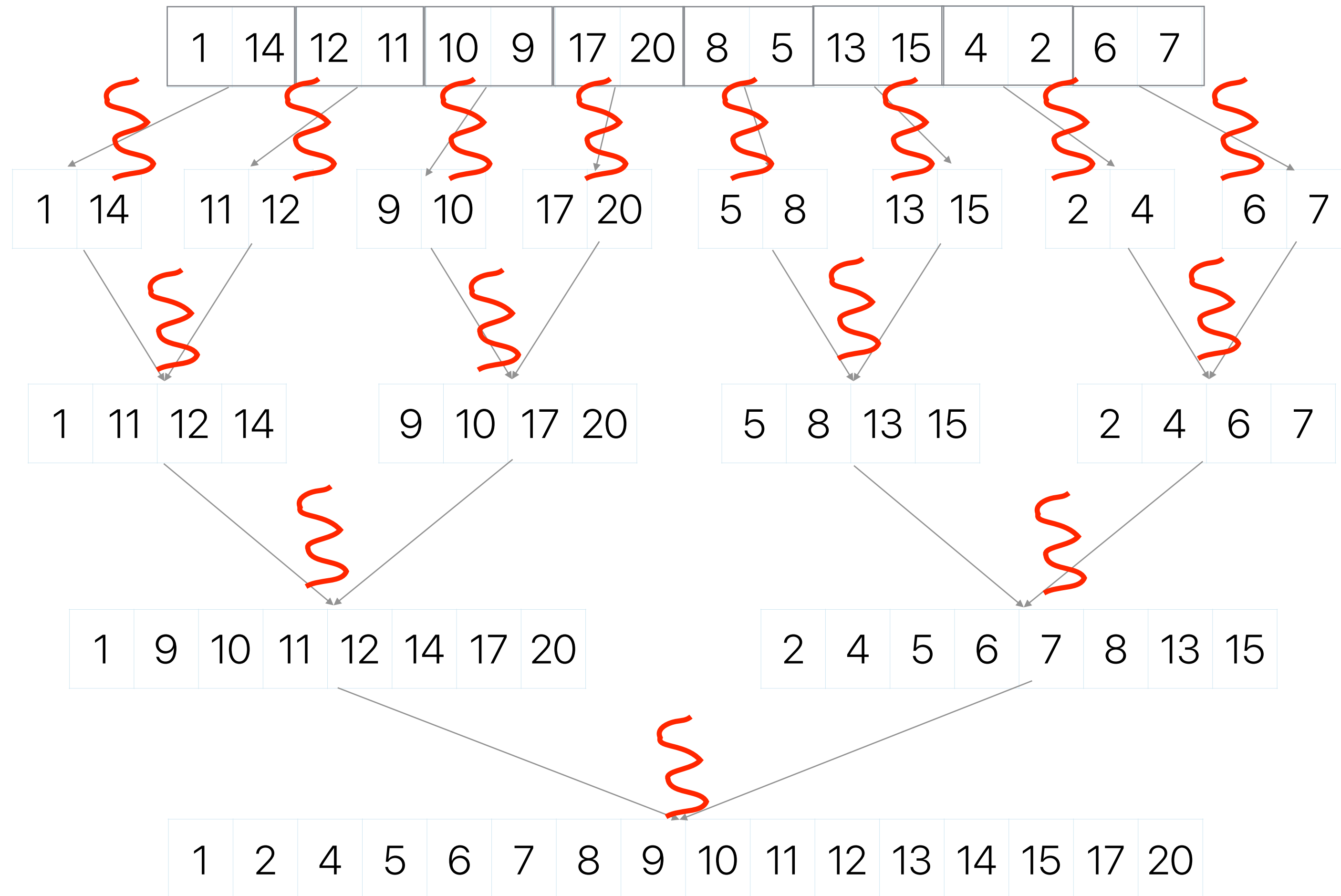
Bitonic Sort
 $O(n \log_2^2 n)$

```
void BitonicSort() {  
  
    int i,j,k;  
  
    for (k=2; k<=N; k=2*k) {  
        for (j=k>>1; j>0; j=j>>1) {  
            for (i=0; i<N; i++) {  
                int ij=i^j;  
                if ((ij)>i) {  
                    if ((i&k)==0 && a[i] > a[ij])  
                        exchange(i,ij);  
                    if ((i&k)!=0 && a[i] < a[ij])  
                        exchange(i,ij);  
                }  
            }  
        }  
    }  
}
```

Merge sort



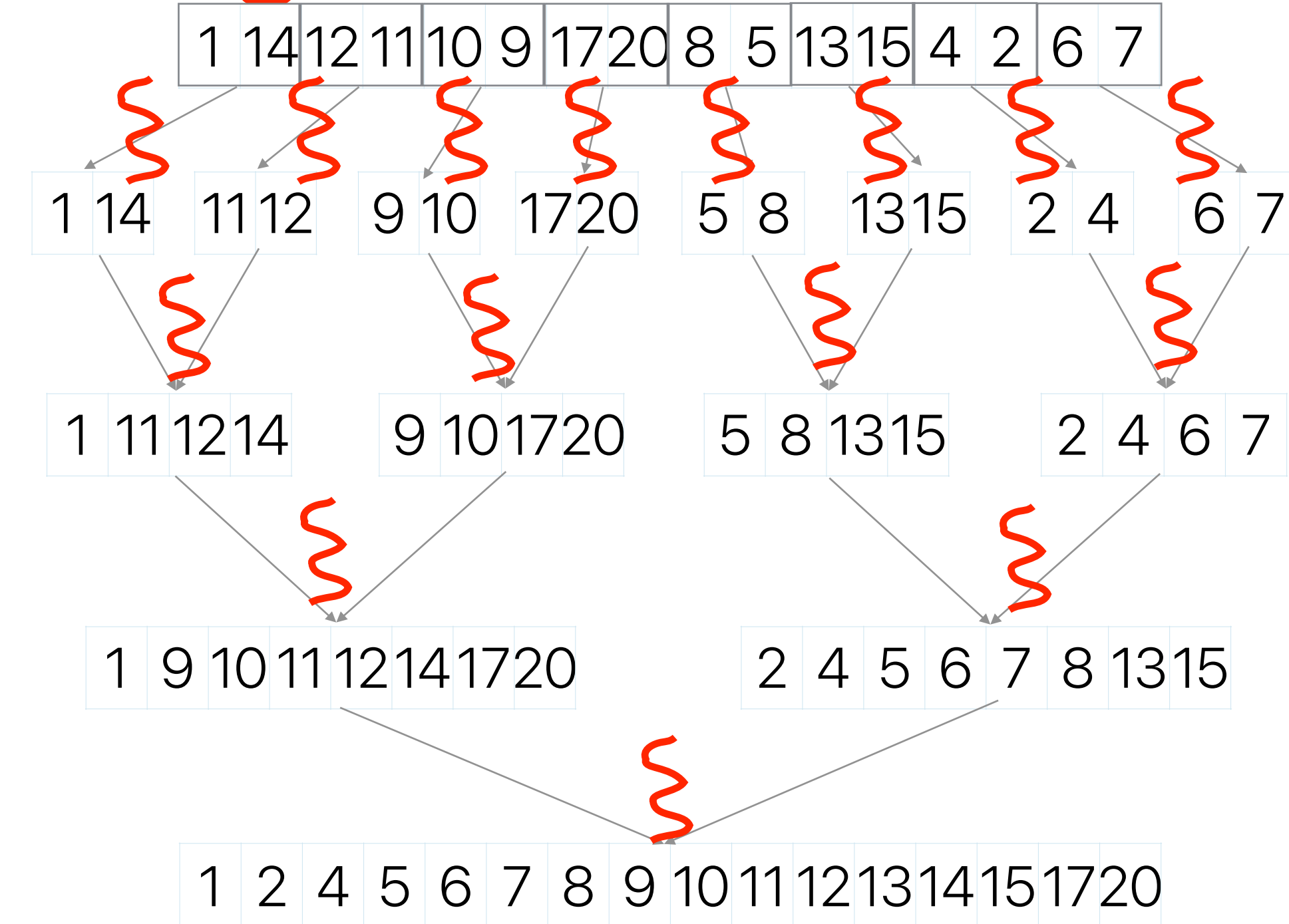
Parallel merge sort



What's the speedup of merge sort using Amdahl's Law

The degree of parallelism is $1, 2, 4, \dots, \frac{n}{2}$
at step $1, 2, 3, \dots, \log_2(n)$

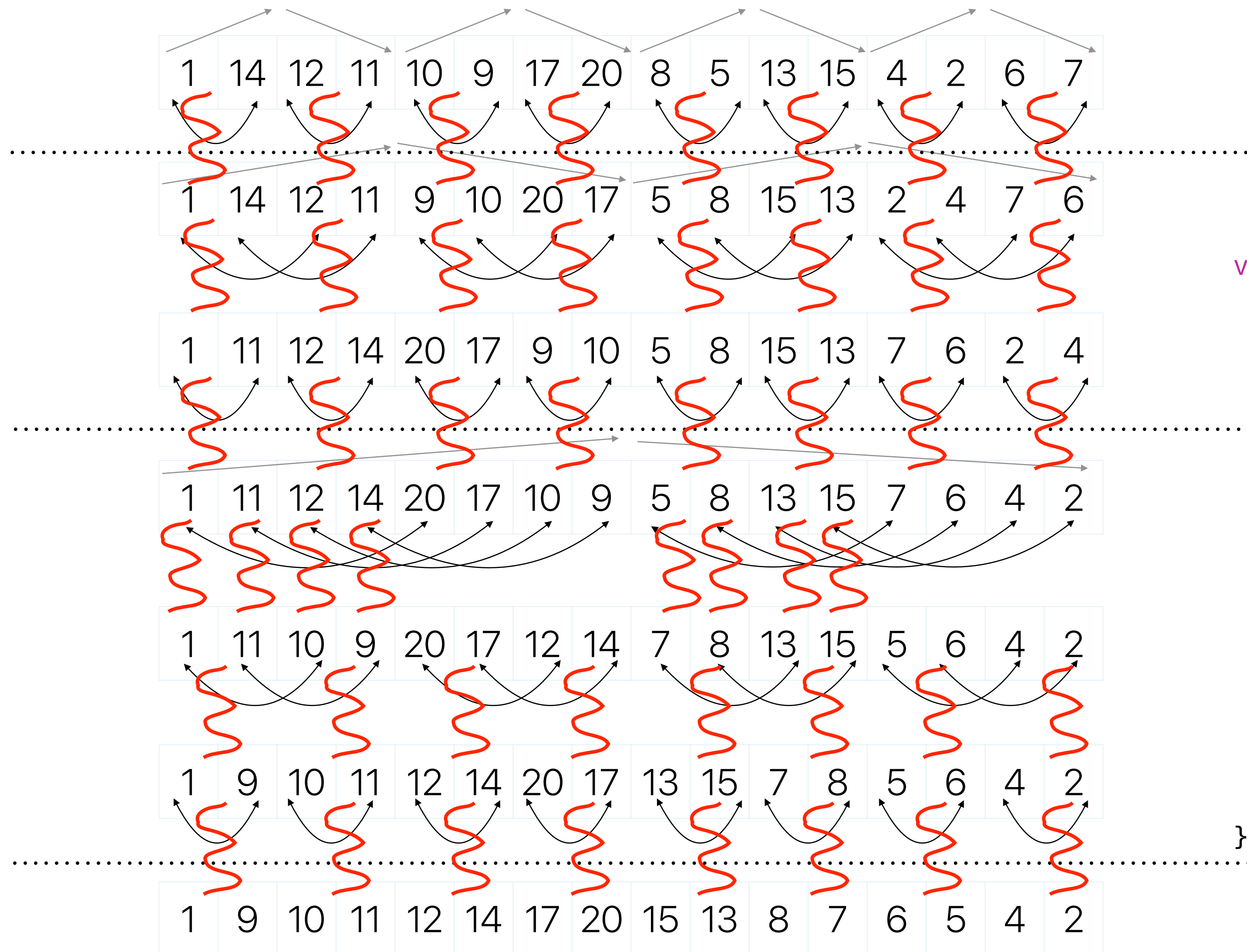
The ideal speedup of each step is $1, 2, 4, \dots, \frac{n}{2}$ or say
 $1, 2, 4, \dots, 2^{\log_2(n)-1}$ if we have **unlimited** parallelism
if we assume equal amount of time in each step in the baseline,
each step is going to take $\frac{1}{\lg(n)}$ portion of time in the baseline, and
the first $\frac{1}{\lg(n)}$ is not parallelizable (i.e., $(1 - x) = \frac{1}{\lg(n)}$)



So the Amdahl's Law's evaluation will become

$$\begin{aligned} \frac{1}{\frac{1}{\lg(n)} + \frac{1}{\lg(n) \times 2} + \frac{1}{\lg(n) \times 4} + \dots + \frac{1}{\lg(n) \times 2^{\lg(n)-1}}} &= \frac{1}{\frac{1}{\lg(n)}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\lg(n)-1}})} \\ &= \frac{1}{\frac{1}{\lg(n)}(1 + 1 - \frac{1}{2^{\lg(n)-1}})} = \frac{1}{\frac{2}{\lg(n)}} = \frac{\lg(n)}{2} \end{aligned}$$

Bitonic sort



```

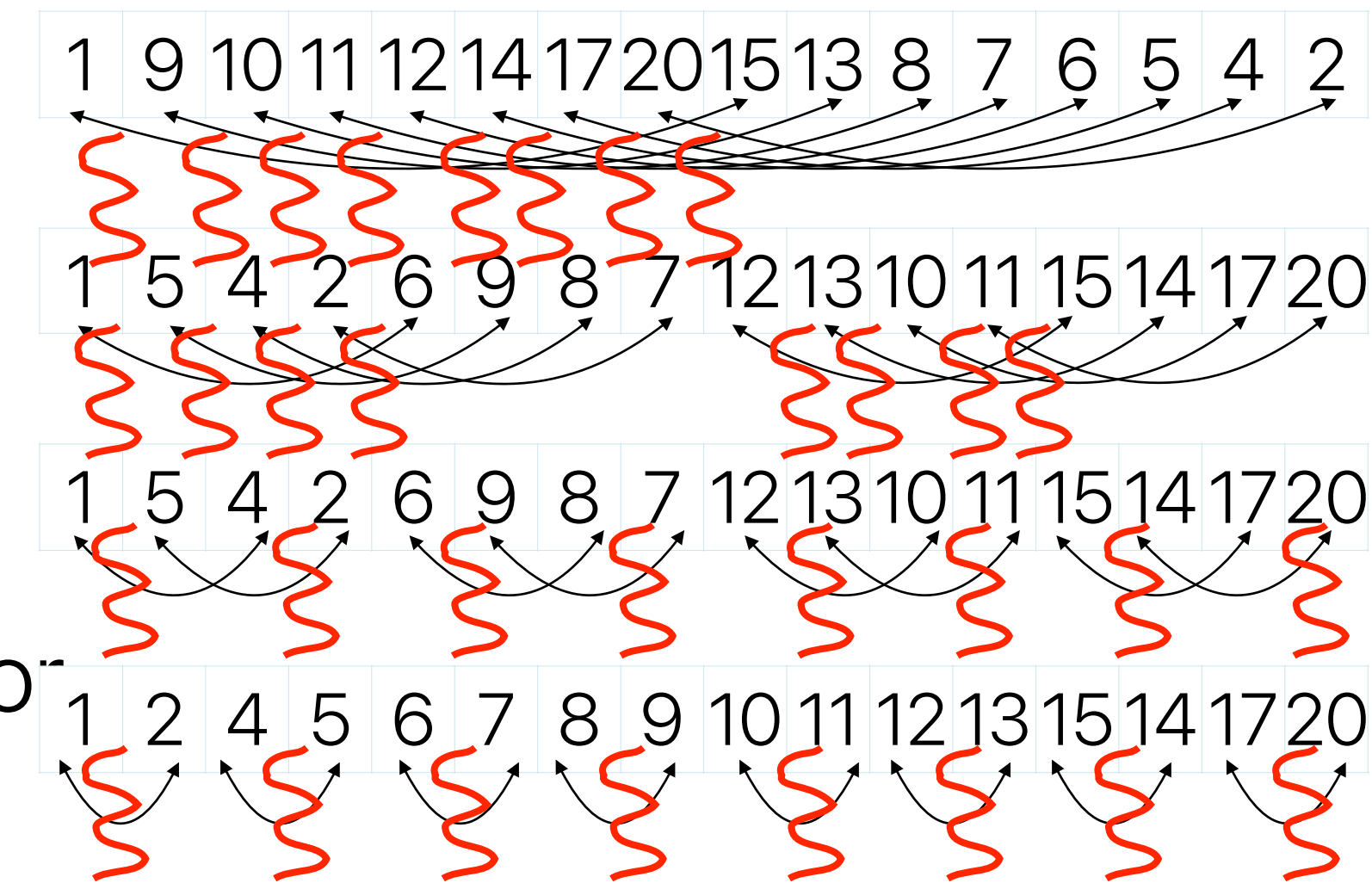
void BitonicSort() {
    int i,j,k;

    for (k=2; k<=N; k=2*k) {
        for (j=k>>1; j>0; j=j>>1) {
            for (i=0; i<N; i++) {
                int ij=i^j;
                if ((ij)>i) {
                    if ((i&k)==0 && a[i] > a[ij])
                        exchange(i,ij);
                    if ((i&k)!=0 && a[i] < a[ij])
                        exchange(i,ij);
                }
            }
        }
    }
}
    
```

What's the speedup of bitonic sort using Amdahl's Law

The degree of parallelism is always $\frac{n}{2}$
at step 1, 2, 3, ..., $\log_2^2(n)$

The ideal speedup of each step is $\frac{n}{2}$ if we have **unlimited** parallelism & for each step of bitonic sort. However, bitonic sort will have $\lg(n)$ x more steps than merge sort.



If the baseline is merge sort — the speedup of using bitonic sort is

$$\frac{\frac{1}{1 \times \lg(n)}}{\frac{n}{2}} = \frac{n}{2 \lg(n)} > \frac{\lg(n)}{2}$$

What if we have only p processors?

For **bitonic sort**, The degree of parallelism is always $\frac{n}{2}$ at step 1, 2, 3, ..., $\log_2^2(n)$, but we have only p processors...

The theoretical speedup of each step is $\max(\frac{n}{2}, p) = p$ since n is very likely to be larger than **p** & assume equal amount of time in each step in the baseline

So the Amdahl's Law's evaluation will become $\frac{1}{\frac{p}{\lg(n)}} = \frac{p}{\lg(n)} = \frac{1024}{30} = 34.1333333$

What about merge sort? $= \frac{1}{\frac{1}{\lg(n)}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{p})}$

What if $p=1024, n=1M=2^{30}$? $= \frac{1}{\frac{1}{30}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{1024} + 20 \times \frac{1}{30})} = 14.862119$

Corollary #4

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{\infty}}$$

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable})}$$

- If we can build a processor with unlimited parallelism
 - The algorithm complexity becomes less important as long as the algorithm can utilize all parallelism
 - That's why bitonic sort or MapReduce works!
- **The future trend of software/application design is seeking for more parallelism rather than lower the computational complexity**

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$
- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$
 - Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 2 — make the common case (the most time consuming case) fast!
$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$
$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$
$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$
$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$
 - Corollary 3 — Optimization has a moving target
 - Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures $Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$

**Is it the end of computational
complexity?**

Corollary #5

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{\infty}}$$

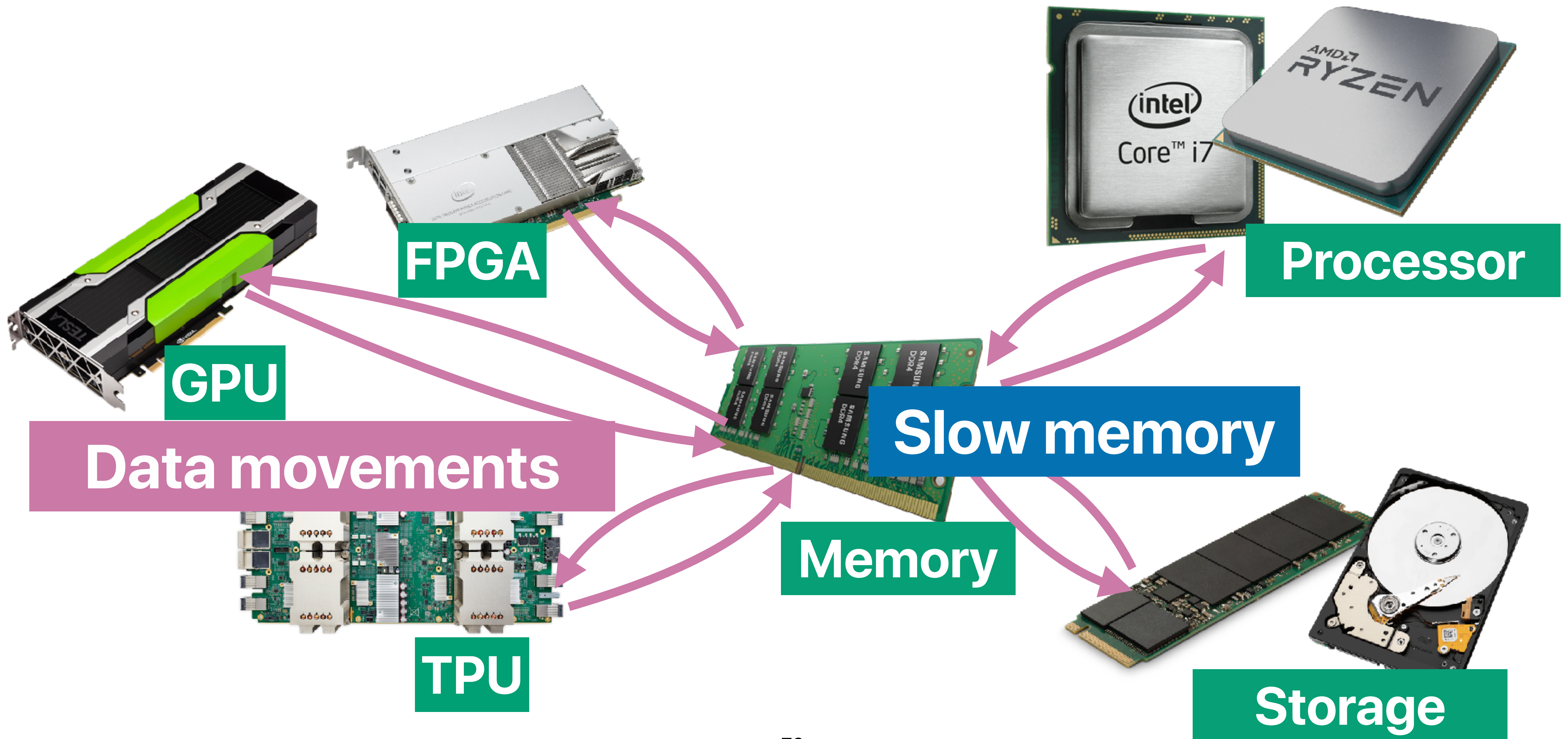
$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable})}$$

- Single-core performance still matters
 - It will eventually dominate the performance
 - If we cannot improve single-core performance further, finding more "parallelizable" parts is more important
 - Algorithm complexity still gives some "insights" regarding the growth of execution time in the same algorithm, though still not accurate

Takeaways: find the right thing to do

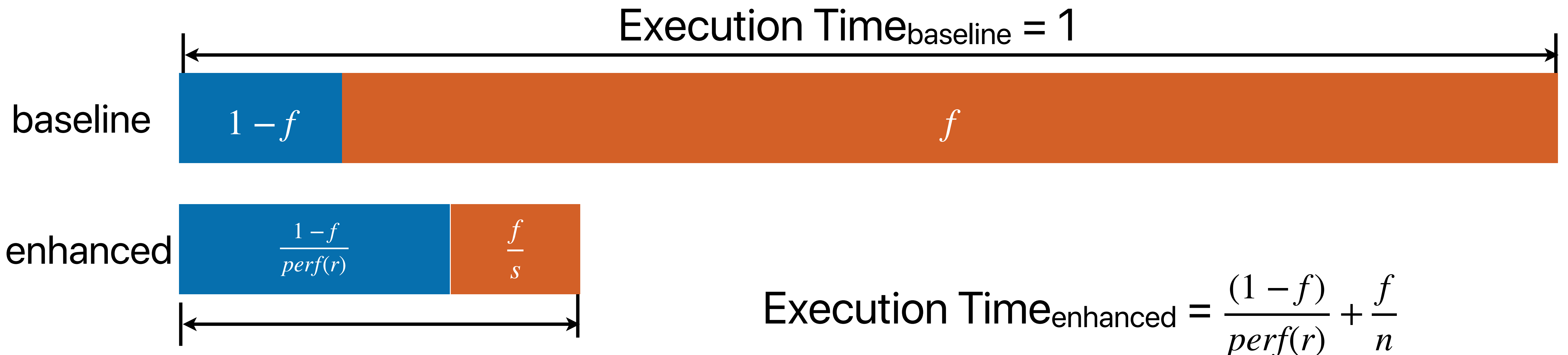
- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$
- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$
 - Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 2 — make the common case (the most time consuming case) fast!
$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$
$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$
$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$
$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$
 - Corollary 3 — Optimization has a moving target
 - Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures $Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$
 - Corollary 5 — Single-core performance still matters $Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$

Most cases, parallelism is not "tax-free"



What if parallelism is not "tax-free"?

- Parallelization Overhead
 - Preparing/exchanging/synchronizing data
 - Additional function calls/control overhead
- The " $1 - f$ " will potentially slowdown by a factor of $perf(r)$



Amdahl's Law considering overhead

$$Speedup_{enhanced}(f, s, r) = \frac{1}{\frac{(1-f)}{perf(r)} + \frac{f}{s}}$$

- r is some other parameter that affects the overhead
 - input size?
 - degree of parallelism?
- $perf(r)$ should be ≤ 1 (i.e., slowdown)
- The overhead may scale differently than the original problem — that's why we introduce " $perf()$ " function

Corollary #6: Don't hurt non-common part too much

- If the program spend 90% in A, 10% in B. Assume that an optimization can accelerate A by 9x, by hurts B by 10x (i.e., a speedup of $\frac{1}{10}$)...

$$\textit{Speedup} = \frac{1}{\frac{(1-f)}{\textit{perf}(r)} + \frac{f}{s}} = \frac{1}{\frac{(1-0.9)}{\frac{1}{10}} + \frac{0.9}{9}} = 0.91 \times$$

Takeaways: find the right thing to do!

- Definition of "Speedup of Y over X" or say Y is n times faster than X:

$$speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

- Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$

- Corollary 2 — make the common case (the most time consuming case) fast!

$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$

- Corollary 3 — Optimization has a moving target

- Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 5 — Single-core performance still matters

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 6 — Don't hurt the non-common case too much

$$Speedup_{enhanced}(f, s, r) = \frac{1}{(1-f) + perf(r) + \frac{f}{s}}$$

Announcement

- Reading quiz due next Tuesday before the lecture
 - We will drop two of your least performing reading quizzes
 - Once you closed, you cannot restart
- Assignment 0 **due tonight** and assignment 1 is released, due next Thursday
 - We cannot help you at the last minute — please start early
 - Watch before you start https://youtu.be/m7OoY8y_Isk
 - Submit to the right item on Gradescope — Gradescope can give you feedback and you're allowed to resubmit
- Check our website for slides/quiz links, Gradescope to submit assignments, piazza for discussions
- Check your grades at https://www.escalab.org/my_grades
- Youtube channel for lecture recordings:
<https://www.youtube.com/c/ProfUsagi/playlists>

Computer Science & Engineering

203

つづく

